

Lab 04

Data Structures

BS DS/CY Fall 2025 Morning/Afternoon

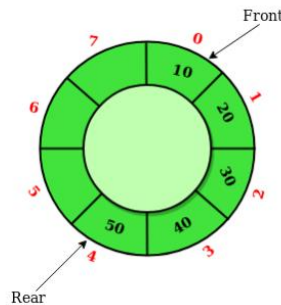
Task 1

10 marks

Circular Queue

A Circular Queue is an extended version of a normal queue where the last element of the queue is connected to the first element of the queue forming a circle.

The operations are performed based on FIFO (First In First Out) principle. It is also called 'Ring Buffer'.



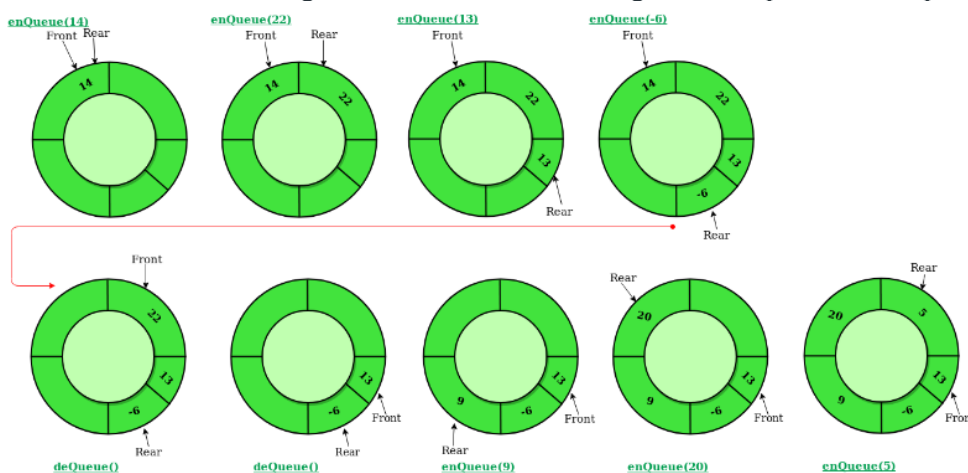
In a normal Queue, we can insert elements until queue becomes full. But once queue becomes full, we can not insert the next element even if there is a space in front of queue.

Operations on Circular Queue:

- **Front:** Get the front item from the queue.
- **Rear:** Get the last item from the queue.
- **enQueue(value)** This function is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at the rear position.
 - Check whether the queue is full – [i.e., the rear end is in just before the front end in a circular manner].
 - If it is full then display Queue is full.
 - If the queue is not full then, insert an element at the end of the queue.
- **deQueue()** This function is used to delete an element from the circular queue. In a circular queue, the element is always deleted from the front position.
 - Check whether the queue is Empty.
 - If it is empty then display Queue is empty.
 - If the queue is not empty, then get the last element and remove it from the queue.

Illustration of Circular Queue Operations:

Follow the below image for a better understanding of the enqueue and dequeue operations.



Working of Circular queue operations

You have to complete the following ADT

```
template<class T>
class Queue
{
private:
    int capacity;
    int rear;
    int front;
    int noOfElement;
    T *data;
public:
    Queue(int cap);
    Queue(Queue<T> &ref);
    void enqueue(T val);
    T dequeue();
    int getNoOfElement();
    int isFull();
    int isEmpty();
    T getFront();
    T getRear();
    Queue<T> operator=(Queue<T> &ref);
    ~Queue();
    void display()
    {
        cout<<"Elements in the circular queue are: ";
        for(int i=0;i<noOfElement;i++)
            cout<<data[(i+front)%capacity]<<" ";
        cout<<endl;
    }
};
```

Test your function using the following driver program

```
int main()
{
    Queue<int> q(5);

    q.enqueue(14);
    q.enqueue(22);
    q.enqueue(13);
    q.enqueue(-6);
    q.display();
    cout<<"Deleted Value = "<<q.dequeue()<<endl;
    cout<<"Deleted Value = "<<q.dequeue()<<endl;
    q.display();

    q.enqueue(9);
    q.enqueue(20);
    q.enqueue(5);
    q.display();
    return 0;
}
```

The output of the following program should be

```
Elements in the circular queue are: 14 22 13 -6
Deleted value = 14
Deleted value = 22
Elements in the circular queue are: 13 -6
Elements in Circular Queue are: 13 -6 9 20 5
```

Task 2

20 marks

A printer has **n print jobs** waiting in a queue. The jobs are arranged from the first job at the front of the queue to the last job at the back.

You are given a **0-indexed integer array pages** of length **n**, where `pages[i]` represents the number of pages remaining in the **ith print job**.

The printer takes exactly **1 second to print one page**. During each turn, the printer prints **only one page** from the job at the front of the queue. If that job still has pages remaining, it is moved to the **end of the queue** immediately. If all its pages have been printed, the job is removed from the queue.

You are also given an integer **k**, representing the position of a particular print job in the original queue.

Calculate the total time required until the print job at position k is completely printed.

Example 1:

Input: Pages = [3, 3, 1, 2], k = 0

Output: 8

Example 2:

Input: Pages = [2,2,2,3,2], k = 3

Output: 11

Task 3

20 marks

You are required to implement a **Browser History Manager** using a **Stack**.

A browser keeps track of the pages visited by a user. Whenever the user visits a new page, the current page is stored in the browser history. When the user selects the **Back** operation, the browser should return to the most recently visited page.

Your program should support the following operations:

1. Visit Page

Visit a new webpage. The current page should be pushed onto the Stack before opening the new page.

2. Go Back

Return to the most recently visited page by removing the top element from the Stack.

3. Display History

Display all pages currently stored in the Stack, starting from the most recently visited page.

4. Exit

Terminate the program.

If the user selects **Go Back** when the Stack is empty, display an appropriate message.

Requirements

- Implement the solution using a **Stack**.
- Use the **Stack implementation provided in the previous task/lab**.
- **Do not use the STL stack class.**
- Each webpage can be represented using a string.
- The program should continue displaying the menu until the user selects **Exit**.
- Properly handle the case when the browser history is empty.

Input Format

The program should repeatedly display the following menu:

1. Visit Page
2. Go Back
3. Display History
4. Exit

For option 1, the user enters the webpage name/address.

For options 2, 3, and 4, no additional input is required.

Sample Input / Output

1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 1 Enter page: Google.com 1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 1 Enter page: YouTube.com 1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 1 Enter page: Wikipedia.org 1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 3 Browser History: Wikipedia.org YouTube.com Google.com	1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 2 Going Back to: YouTube.com 1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 2 Going Back to: Google.com 1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 3 Browser History: No previous pages available. 1. Visit Page 2. Go Back 3. Display History 4. Exit Enter choice: 4 Exiting Browser History Manager...
---	--

Task 4 : Restaurant Order Processing Simulator

50 marks

In this task, you are required to simulate a **Restaurant Order Manager**. The Order Manager is responsible for processing customer orders one by one.

The **Order Manager** should take a list of **Orders** and add them to a Queue. Once all the orders have been added, your program should start processing them.

Before processing each order, the program should display the **order ID, customer/order name, and remaining preparation time**.

The system should process the order at the front of the Queue. Processing an order means that the order remains at the front of the Queue while its **preparation time** is reduced.

The restaurant uses a provided **time quantum**. After the specified time quantum has elapsed, the system must stop processing the current order. If the order is not completed, it should be removed from the front of the Queue and added to the **end of the Queue**, with its remaining preparation time updated.

If an order's remaining preparation time becomes **0**, it should be completely removed from the Queue. At this point, the program should display a message indicating that the order has been completed.

For example:

Order (101, Ali) completed preparation

You must remember that this is a **simulator**. Therefore, you must define your own timer that increments in unit steps. The simulation should continue until **all orders have been completely prepared and the Queue becomes empty**.

Hints

Design a class **Order** with the following data members:

- orderId
- customerName
- preparationTime

and appropriate methods to access and update the order information.

Create a **circular Queue** and add all the orders to it.

Process the **Order at the front of the Queue**. After the specified time quantum has passed, remove the order from the front and add it to the end of the Queue if it still has preparation time remaining.

If an order has **0 preparation time remaining**, remove it completely from the Queue.

Repeat this process until the Queue is empty.

*You have to create the following files and will have to code all the classes listed in these files.

Order.h

```
class Order {
private:
    int orderId;
    string customerName;
    int preparationTime;
public:
    Order() {
        orderId = 0;
        customerName = "";
        preparationTime = 0;
    }
    Order(int id, string name, int time);

    int getId();
    string getName();
    int getPreparationTime();

    void setPreparationTime(int t);
};
```

OrderManager.h

```
class OrderManager {  
  
private:  
    Queue<Order> q;  
    int timeQuantum;  
  
public:  
  
    OrderManager(Order arr[], int len, int tq);  
  
    void processOrders();  
};
```

Queue // use the task 3 queue

main.cpp

```
int main()  
{  
    Order arr[] = {  
        Order(101, "Ali", 20),  
        Order(102, "Sara", 5),  
        Order(103, "Ahmed", 30),  
        Order(104, "Usman", 2)  
    };  
  
    OrderManager manager(arr, 4, 5); // time quantum = 5  
    manager.processOrders();  
    return 0;  
}
```

The output of this program should be:

Processing Order (101, Ali) for 5 units
Processing Order (102, Sara) for 5 units

Order (102, Sara) completed preparation.

Processing Order (103, Ahmed) for 5 units
Processing Order (104, Usman) for 2 units

Order (104, Usman) completed preparation.

Processing Order (101, Ali) for 5 units
Processing Order (103, Ahmed) for 5 units
Processing Order (101, Ali) for 5 units
Processing Order (103, Ahmed) for 5 units
Processing Order (101, Ali) for 5 units

Order (101, Ali) completed preparation.

Processing Order (103, Ahmed) for 5 units
Processing Order (103, Ahmed) for 5 units
Processing Order (103, Ahmed) for 5 units

Order (103, Ahmed) completed preparation.